Look-Ahead Genetic Programming for Uncertain Capacitated Arc Routing Problem

Jordan MacLachlan and Yi Mei
School of Engineering and Computer Science
Victoria University of Wellington
Email: {maclacjord, yi.mei}@ecs.vuw.ac.nz

***Abstract*—Genetic Programming Hyper-Heuristic (GPHH) has been successfully applied to evolve routing policies for the Uncertain Capacitated Arc Routing Problem (UCARP). However, the current GPHH approaches have a limitation that they only consider myopic information of the current decision step. In this paper, we proposed incorporating look-ahead information to the decision process of GP-evolved routing policies. We designed a number of potentially promising *chains* of candidate tasks, and expand the candidate task pool to consider both the single tasks and task chains. This way, the routing policy can consider the look-ahead information incorporated in the considered task chains. The proposed GP with Chain Policies (GPCP) was compared with the standard GPHH on a range of UCARP instances, and the results showed that the task chains can improve the effectiveness of the routing policies sometimes. The better performance of a routing policy largely depends on whether it can balance the selections of single tasks and task chains, and whether it can stick to the whole selected chain rather than only the first task of the chain. In addition, there are some abnormal runs with serious overfitting issue that we will address in our future work.**

I. Introduction

The Capacitated Arc Routing Problem (CARP) [1], [2] is of great commercial and scholarly focus due to its applicability to many real-world problems, such as civil refuse collection [3], [4], [5], [6], disaster recovery and cleanup [7], [8], [9], and snow management [10], [11], [12]. Briefly speaking, CARP considers a graph of arc-connected nodes, and a fleet of vehicles with a limited capacity. The aim is to serve all (capacity depleting) arc-tasks by the vehicles with the minimal total cost, while a set of constraints are enforced. For example, no vehicle can exceed its capacity, and the arc-tasks cannot be shared among different vehicles.

Traditionally, CARP assumes full knowledge of a static state. However, randomness pervades many problems CARP has been used to represent. For example, one may not precisely gauge how much garbage a household will produce (especially around holidays), or exactly how much snow needs to be cleared after a blizzard. Nor can they foresee traffic accidents, or otherwise spontaneous traffic disruptions. The Uncertain CARP (UCARP) was proposed [13], [14] to step CARP towards reality by introducing uncertainty.

In addition to the NP-hardness of CARP [1], the uncertain environment induces extra challenges. For example, given a pre-designed set of routes, the actual demand of a task may exceed the vehicle's remaining capacity upon arrival (so-called *route failure*) during the execution process. In this case, a *recourse operator* is needed to repair the remaining routes. A typical recourse operator is to directly return to the depot to replenish capacity, and then come back to continue the failed service. Such recourse operations can incur significant extra cost compared to the pre-planned route.

At a high level, two main methods have been applied to solve UCARP effectively to date; the formation of robust routes with recourse [15], and an online decision making process, i.e. routing policies [16], [17], [14], [18]. The robust optimisation method obtains a pre-designed set of routes, and adjusts them by recourse policies when necessary. In the routing policy-based method, when a vehicle becomes idle, the associated routing rule is used to select the next task among the remaining tasks for an idle vehicle to serve. This paper focuses on the routing policy-based method, since a routing policy can make the routing decision based on the up-to-date information, and thus can react to the uncertain environment effectively. An example of a routing policy is the path scanning heuristic [1], [19], which selects the nearest task to serve next, and uses five different tie-breaking rules.

Genetic Programming (GP) has been successfully applied to UCARP [16], [17], [20] to evolve much more effective routing policies than those manually designed (e.g. path scanning). However, the current GP-evolved routing policies are myopic, considering only the local information of each candidate immediate next task [21], [14]. Such myopic routing policies can make sub-optimal decisions, such as assigning a next task to a vehicle that could have been much better to be assigned to another vehicle which will become idle later. On the other hand, although rollout approaches can explore more steps ahead, the search space and computational complexity grows exponentially as the number of steps increases. As a result, a complete rollout approach is too time consuming and cannot be afforded in practice.

To address the above issues, this paper proposes a novel GP with look-ahead features to evolve more effective routing policies for UCARP. Specifically, we have the following research objectives:

- To design a new type of routing policy that considers task *chains*, i.e. a sequence of tasks, rather than a single task.
- To design new look-ahead features specifically for task chains to be considered by the *chain policy*, and embed them into the GP terminal set.

- To develop a GP algorithm to evolve the chain policies.
- To verify the effectiveness of the GP-evolved chain policies on a wide range of UCARP instances, and analyse the effectiveness of the look-ahead features.

The rest of this paper is organised as follows. Section II defines the problem, and outlines the related works and GPHH method. Section III details the chain policy algorithm, and the new chain policy features. Section VI presents and discusses the outcome of the proposed algorithm. We conclude in Section VII with a summary, and an overview of our projected future work.

II. Background

A. The Uncertain Capacitated Arc Routing Problem

A UCARP instance is defined on a connected directed graph $G = (V, A)$. Any undirected edge can be converted into two mutually inverse directed edges, i.e. *arcs*. For the sake of simplicity, we denote the inverse arc of an arc a to be a^{-1}. The pair of arcs a and a^{-1} associated with each edge $\in E$ share three non-negative attributes: a random demand $\tilde{d}(a)$, a random traversal cost $\tilde{c}_t(a)$ and a deterministic serving cost $c_s(a)$. An arc with an expected positive demand is also called a *task*. Let the set of tasks to be A_T, and non-required tasks to be A_N, we have $A_T \cup A_N = A$ and $A_T \cap A_N = \emptyset$. A fleet of m vehicles are located at the depot $v_0 \in V$ to serving the tasks. Each vehicle has a limited capacity Q, meaning that a single vehicle cannot serve all the tasks. The goal is to minimise the total cost (traversal plus serving costs) of the routes of the vehicles so that:

- Each vehicle starts from the depot, and returns to the depot after finishing all the services;
- Each task is served by exactly one vehicle. In case of route failure, the vehicle can go back to the depot in the middle of the service to replenish, and then return to complete the failed service;
- The total actual demand of the tasks served by a vehicle between two depot visits cannot exceed its capacity Q.

A UCARP instance contains a number of random variables (e.g. $\tilde{d}(a)$ and $\tilde{c}_t(a)$). Given an instance I, by sampling a value for each random variable (e.g. random demand $\tilde{d}(a)$ to sample $d_\xi(a)$, random traversal cost $\tilde{c}_t(a)$ to sample $c_{t,\xi}(a)$), we can generate a *sample* I_ξ of the instance I. Note that a UCARP instance can have infinite number of different samples.

A UCARP instance sample is similar to a static CARP instance, except that the actual value of each random variable is unknown until it is realised. For example, the actual demand of a task is unknown until a vehicle completes it.

B. Related Works

Many methods effective in a static environment are limited by their inability to (i) scale with instance size, and/or (ii) handle uncertainty (i.e. they require an unchanging state). Solutions to uncertain problems are usually non-optimal. Instead the aim is to minimise an objective cost measure (although 'objectivity' can often be called into question [22]). Many different aspects of uncertainty have been considered, both in vehicle routing problem (a node-routing counterpart) [23], [24], [25] and ARP [13], with little consensus on the most realistic representation. UCARP [13] has drawn several such elements together.

In a broad sense, approaches to solving UCARP can be categorised as as follows [26], [27].

- **Proactive**: The proactive approach typically optimises a solution based on the prediction about the environment, and obtain robust solutions that can retain reasonably good quality (after recourse if necessary) in all the possible environments. The prediction can be represented as a random distribution or a set of training instances.
- **Reactive**: The reactive approach typically does not optimise a solution in advance, but builds the solution on-the-fly during the execution process. This way, the up-to-date information about the environment (e.g. actual demand of the tasks that have been completed, and the actual remaining capacity of each vehicle) can be taken into account to make a more effective decision. Routing policy is a typical reactive approach.
- **Predictive-Reactive**: The predictive-reactive approach is a hybrid approach of the predictive and reactive approaches. It typically solves the dynamic problem in a rolling horizon way. In each time window, the approach takes the new environment into account, and re-optimises the current partial solution using time-efficient heuristic methods.

Below is detailed a handful of interesting approaches to solving some element of uncertainty in CARP. Few true predictive reactive approaches have been proposed for uncertain routing problems.

1) Proactive: Gonzalez et al. [28] developed a 'simheuristic' method of solving CARP with stochastic demands. First, initial solutions are generated using a static method [29]. Then, a set of Monte Carlo simulations are iterated to determine the risk level of each route, i.e. what the likelihood of route failure is. This information can then be utilised to manipulate the constructed routes, if need be.

For UCARP, Wang et al. [15] proposed a very effective Estimation of the Distribution Algorithm with Stochastic Local Search (EDASLS). The method designs *robust* solutions (those that incur failures at little additional cost) with built-in recourse operations to handle significant perturbations in the state, such as route failures. The published EDA component of EDASLS, however, relies on knowing the demand of each task in full, prior to arrival - an assumption reactive methods cannot make.

2) Reactive: Toklu et al. [30] presented an Ant Colony system that utilised the pheromone tracing ability of a previous static work to produce initial solutions, which are then modified further using a simple 3-opt operator. This method only took into account uncertain travel times.

UCARP has been further studied in a reactive sense. [16] proposed a genetic programming hyper-heuristic (GPHH) to

evolve *routing rules* as priority functions to select idle vehicles' next tasks, when needed. This was expanded upon by [14] to create a multi-vehicle simulation that allowed for the simultaneous construction of each route in parallel. Further works about GPHH for UCARP include considering knowledge transfer among different UCARP scenarios [31], [32] and learning a group of diverse routing policies rather than a single complex one [33], [34]. Considering collaborations between vehicles [18] can significantly improve performance, however requires the modification of several constraints, making direct comparison to existing algorithms questionable.

Rollout algorithms, whereby at any given decision point you explore several possible states ahead of you to hypothesise which is the best option, is an interesting field in uncertain optimisation. Bertazzi and Secomandi [35] solved the VRP with stochastic demands by 'planning' the entire route ahead via rollout. Ulmer et al. [36] used an approximate dynamic programming method to achieve a similar end; forward solution planning. However, this particular method was limited by using a single uncapacitated vehicle.

3) Predictive-Reactive: To the best of our knowledge, there is only one existing predictive-reactive method to solving UCARP, which is the Solution Policy Co-evolver (SoPoC) [37]. The algorithm optimises both a robust solution via an Estimation of the Distribution method and a GPHH-evolved recourse operator, which decided whether to return to the depot, or continue to the next task during the execution process. The authors found that in certain circumstances, it was advantageous for the vehicle to trigger an early refill, such as when the vehicle was already close to the depot and low on capacity. SoPoC was found to significantly outperform EDASLS on several gdb and val instances. However, as SoPoC is an iterated improvement of EDASLS (and GPHH), we currently maintain EDASLS as the existing state-of-the-art.

C. Genetic Programming Hyper-Heuristic for UCARP

Algorithm 1 shows the outline of GPHH for UCARP [17], [14], [18]. The individuals in Algorithm 1 are routing policies [38] to solve UCARP instance samples. The evaluation of a routing policy is performed by applying the policy to a number of training instance samples. Specifically, at each decision point, i.e. when a vehicle becomes idle, the routing policy calculates the priority value of each candidate task (e.g. unserved feasible task), and selects the most prior task based on the priority values. More details can be found from [14], [18].

Algorithm 1 The outline of a GPHH for UCARP

Randomly initialise a population of routing policies;
while stopping criteria is not met **do**:
  Evaluate routing policies in the current population;
  while new population is not full **do**:
    Select routing policies to be used;
    Generate new policies via genetic operators;
    Add resultant policies to the new population;

III. Routing Policy for Task Chains

The newly proposed *chain policy* is a type of routing policy that considers *task chains* rather than only single tasks. Here, a task chain is simply a sequence of tasks. A task chain with a length of k (i.e. a sequence of k tasks) is said to be a ℓ_k chain ($k > 1$). ℓ_1 denotes a length-1 chain.

The decision making process for building the solution using a chain policy is described in Algorithm 2.

Algorithm 2 The chain-policy decision making process

Input: A UCARP instance sample I_ξ, the chain policy $h^c(\cdot)$
Output: The built solution S_{I_ξ}
1: All the vehicles are at the depot, all the tasks unserved, S_{I_ξ} contains an empty route for each vehicle;
2: **while** not all tasks served **do**
3: Find the earliest idle vehicle veh^*;
4: Find all the feasible unserved tasks Ω;
5: **Generate the candidate chains** Ω^c **(Algorithm 4);**
6: **for each** chain $C \in \Omega^c$ **do**
7: Calculate its priority value by $h^c(C)$;
8: Select the next chain $C^* = \arg\min\{h^c(C)|C \in \Omega^c\}$;
9: Send veh^* to serve **the first task of** C^*, update S_{I_ξ};
10: **return** S_{I_ξ};

Algorithm 3 The GPHH decision making process

Input: A UCARP instance sample I_ξ, the routing policy $h(\cdot)$
Output: The built solution S_{I_ξ}
All the vehicles are at the depot, all the tasks unserved, S_{I_ξ} contains an empty route for each vehicle;
while not all tasks served **do**
  Find the earliest idle vehicle veh^*;
  Find all the feasible unserved tasks Ω;
  for each candidate task $t \in \Omega$ **do**
  Calculate its priority value by $h(t)$;
  Select the next task $t^* = \arg\min\{h(t)|t \in \Omega\}$;
  Send veh^* to serve t^*, update S_{I_ξ};
return S_{I_ξ};

The main difference between Algorithm 2 and the commonly used decision making processes based on single tasks is between lines 5–9, which takes task chains into account. Note that after selecting the next task chain, the idle is only required to serve the first task in the chain. This way, we offer additional flexibility of the decision making process so that the vehicle can still change its route in the middle of the chain if unpredicted events occur, making it undesirable to keep the current task sequence.

Algorithm 2 has two main design issues. The first is the generation of candidate task chains (line 5), and the second is the priority calculation of each candidate task chain (line 7).

A. Generation of Candidate Chains

Given a set of n candidate tasks on a fully connected graph, the number of all possible ℓ_k task chains is $P_k^n = \frac{n!}{(n-k)!}$. In general, the number of ℓ_k chains rapidly increases as k increases, leading to much larger candidate set. Each traditional candidate task can be seen as an ℓ_1 chain, and there are n such single-task chains. On the other hand, the rollout approach can be seen to consider all the ℓ_n chains, and the size

of the candidate set is $n!$. Therefore, it is important to design a proper candidate task set to balance search space size and computational complexity.

To achieve this purpose, we develop a new candidate chain generation method that takes a promising subset of ℓ_k chains to reduce the computational complexity without losing much promising regions in the search space. It is described in Algorithm 4. Initially, it obtains the set Ω_1 of the candidate single tasks that are expected to be feasible, i.e. its expected demand does not exceed the remaining capacity (lines 1–5). The set of candidate chains Ω_c is initalised with all these feasible single-task chains, to ensure Ω_c is at least a superset of the standard GPHH candidate set. After that, it expands the ℓ_1 chains in Ω_c as follows. For each longest chain, it examines all the candidate tasks that do not already exist in the chain, where insertion is not expected to violate the capacity constraint (line 11). If the task is immediately connected to the end of the chain (the last node of the chain equals the head node of the task - line 13), then a new chain is generated by inserting the task to the end of the chain.

Algorithm 4 The generation of candidate task chains.

Input: The candidate single tasks Ω, the current vehicle veh^*, the maximal length of the task chain K
Output: A set of candidate chains Ω^c

1: Get the remaining capacity of veh^*: $\bar{Q}$;
2: Set $\Omega_1 = \emptyset$;
3: **for each** single task $a \in \Omega$ **do**
4: **if** $\mathtt{E}[\tilde{d}(a)] \leq \bar{Q}$ **then**
5: $\Omega^c = \Omega^c \cup a$, $\Omega_1 = \Omega_1 \cup a$;
6: Set $\Omega^c = \Omega_1$, $k = 1$;
7: **while** $k < K$ **do**
8: **for each** ℓ_k chain $C \in \Omega^c$ **do**
9: Get the tail node C[end] of the last task of C;
10: **for each** task $a' \in \Omega$ **do**
11: **if** $a' \notin C$ and $\mathtt{E}[\tilde{d}(a')] + d(C) \leq \bar{Q}$ **then**
12: Get the head node $head(a')$ of a';
13: **if** $head(a') = C$[end] **then**
14: Generate a new chain $C' = [C, a']$;
15: $\Omega^c = \Omega^c \cup C'$;
16: $k = k + 1$;
17: **return** Ω^c;

Algorithm 4 uses two filters to reduce the number of candidate chains. First, it rules out the chains that are expected to violate the capacity constraint (its expected total demand exceeds the remaining capacity). Second, it considers only the chains with tasks that are immediately connected to each other. Such chains are more likely to reduce the total cost, as these tasks can be served without incurring any extra traversal cost.

Note: the true implementation of this method is recursive. An iterative alternative is presented for claritive means.

B. Priority Calculation

The priority function of a task chain is essentially a mathematical expression of the features of the chain. When evolving single-task routing policies using GP, the terminal set includes the features of a single task (e.g. the expected demand, serving cost, and the travel cost from the current location).

For calculating the priority of a task chain, it is necessary to define new features for task chains. In this paper, we consider a chain as a "meta-task". Specifically,

- the head node of the chain is the head node of the first task of the chain;
- the tail node of the chain is the tail node of the last task of the chain;
- task-specific features of the chain are summed across all tasks in the chain.

This way, most of the existing single-task features can be directly applied to task chains.

In addition, we develop two new features specifically for the chains with no less than 2 tasks. The two features are described as follows.

- `InOtherChain`(`IOC`): this feature indicates whether the first task of the candidate chain has already been selected by other vehicles. It returns 1 if so, and 0 otherwise.
- `InThisChain`(`ITC`): this feature indicates whether the first task of the candidate chain is in the current chain of the current vehicle. It returns 1 if so, and 0 otherwise.

The two new features attempt to routes overlapping, i.e. where different vehicles select the same chain (or task).

C. Summary

The chain policy can be seen as a hybrid technique between the traditional routing policy and a rollout approach. Compared with the traditional routing policy, the chain policy considers more look-ahead information by exploring more steps ahead. Compared with the rollout approaches, the chain policy reduces the search space by identifying the potentially most promising future branches, rather than expanding all possible branches. For UCARP, this is achieved by employing domain knowledge, e.g. the adjacency among the candidate tasks and the vehicle location.

Overall, we expect that the chain policy can perform much better than the routing policy with slight overhead. Compared with the rollout approach, it may achieve comparable results, but can be much faster.

IV. GP for Evolving Chain Policies

The GP algorithm for evolving chain policies (GPCP) is described in Algorithm 5. It follows the standard GP process, except that the fitness evaluation of a chain policy (line 4–6) is done using the chain policy decision making process (Algorithm 2). Specifically, the chain policy is applied to each training instance sample to generate a solution by Algorithm 2. Then the fitness is defined as the mean total cost of the generated solutions.

V. Experiment Design

To determine the efficacy of the chain policy method, we compare with the baseline GPHH [17], on two commonly used UCARP datasets: *Ugdb* and *Uval*. Both datasets contain artificially designed, relatively small (< 100 tasks) road networks. The column values $|V|$, $|E|$ and m in the subsequent result tables respectively correspond to the number of vertices,

Algorithm 5 The proposed GPCP.

Input: The UCARP training instance samples Ξ
Output: The chain policy $h^*(\cdot)$
Initialise a population of chain policies $\mathcal{H}$;
while stopping criteria is not met **do**
  for each chain policy $h \in \mathcal{H}$ **do**
    for each sample $I_\xi \in \Xi$ **do**
      Build solution $S_{I_\xi}(h)$ by Algorithm 2;
    $fit(h) = \frac{1}{|\Xi|}\sum_{I_\xi \in \Xi} C(S_{I_\xi}(h))$;
  Set new population $\mathcal{H}' = \emptyset$;
  while $\mathcal{H}'$ is not full **do**
    Select parents from $\mathcal{H}$ by tournament selection;
    Generate new chain policies using GP crossover/mutation/reproduction operators;
    $\mathcal{H} = \mathcal{H}'$;
return the best chain policy in $\mathcal{H}$;

the number of edges and the number of vehicles in a given instance.

Each algorithm undergoes a distinct period of training and of testing. During training, we evolve routing policies using 5 UCARP instance samples. The routing policy's fitness is defined as the average total cost across these 5 samples. During testing, the best trained individual is applied to 500 unseen test instance samples.

The compared algorithms share the same parameter settings as shown in Table I. The function set is restricted to the standard mathematical operators (with protected division), plus the min & max functions: $\{\times, \%, +, -, \min, \max\}$, while the terminal set is as defined in Table II.

TABLE I
THE PARAMETER SETTINGS FOR THE COMPARED ALGORITHMS.

Parameter	Value	Parameter	Value
Population size ϱ	1024	Generations Γ	51
Tournament size	7	Crossover rate	0.8
Mutation rate	0.15	Reproduction rate	0.05
Maximal depth	8		

TABLE II
THE TERMINAL SET EMPLOYED BY THE CHAIN POLICY FOR UCARP

Notation	Description
CFH	Cost From Here (the current node) to the candidate task.
CFR1	Cost From the closest alternative Route to the task.
CR	Cost to Refill (from the current node to the depot).
CTD	Cost from the candidate task To the Depot.
CTT1	Cost from the candidate task To its closest unserved Task.
DEM	DEMand of the candidate task.
DEM1	DEMand of the closest unserved task to the candidate task.
FRT	Fraction of the Remaining (unserved) Tasks .
FUT	Fraction of the Unassigned Tasks.
FULL	FULLness of the vehicle (current load over capacity).
RQ	Remaining Capacity of the vehicle.
RQ1	Remaining Capacity for the closest alternative route.
SC	Serving Cost of the candidate task.
ERC	a random constant value.

TABLE III
THE TEST PERFORMANCE OF THE GPHH AND GPCP (WITH $K = 2, 3$) METHODS ON THE *gdb* INSTANCES.

Instance	$\|V\|$	$\|E\|$	m	GPHH	GPCP$_{K2}$	GPCP$_{K3}$
Ugdb1	12	22	5	349.55(7.26)(= =)	352.55(20.06)	355.47(18.43)
Ugdb2	12	26	6	374.86(10.26)(- =)	378.18(5.01)	**375.44(8.26)**
Ugdb3	12	22	5	307.95(6.69)(= =)	305.64(2.14)	307.27(3.96)
Ugdb4	11	19	4	326.24(14.78)(- -)	329.44(6.28)	330.54(5.58)
Ugdb5	13	26	6	426.42(14.32)(= =)	427.38(8.84)	431.52(30.82)
Ugdb6	12	22	5	343.56(7.36)(= =)	347.42(20.34)	348.18(17.30)
Ugdb7	12	22	5	355.88(4.80)(= =)	356.09(3.44)	356.97(3.69)
Ugdb8	27	46	10	433.03(7.08)(= =)	429.75(8.14)	431.07(5.25)
Ugdb9	27	51	10	393.57(10.08)(+ =)	**387.62(9.49)**	393.88(7.90)
Ugdb10	12	25	4	292.54(6.82)(= =)	292.71(5.49)	293.94(8.67)
Ugdb11	22	45	5	437.10(8.05)(= =)	436.54(8.85)	439.10(7.47)
Ugdb12	13	23	7	610.19(13.08)(= =)	620.11(87.32)	**613.79(14.75)**
Ugdb13	10	28	6	580.48(8.38)(- -)	590.97(41.51)	584.92(6.37)
Ugdb14	7	21	5	107.10(1.17)(= =)	107.19(1.39)	107.43(1.76)
Ugdb15	7	21	4	58.18(0.25)(= =)	58.70(3.29)	58.40(1.12)
Ugdb16	8	28	5	134.62(0.17)(= =)	134.63(0.27)	134.61(0.18)
Ugdb17	8	28	5	91.27(0.23)(= =)	91.27(0.03)	**91.21(0.09)**
Ugdb18	9	36	5	167.56(1.95)(= -)	168.86(5.27)	168.51(1.30)
Ugdb19	8	11	3	61.20(1.16)(= =)	61.26(1.22)	61.34(1.49)
Ugdb20	11	22	4	127.01(1.39)(- -)	129.08(3.75)	127.88(1.49)
Ugdb21	11	33	6	165.82(3.27)(= =)	165.56(2.23)	166.89(3.33)
Ugdb22	11	44	8	210.81(3.53)(= =)	210.57(1.59)	210.26(1.81)
Ugdb23	11	55	10	250.40(2.83)(- -)	251.85(2.12)	252.04(2.68)

VI. RESULTS AND DISCUSSIONS

The test performance of the compared algorithms are shown in Tables III and IV. The tables show the performance of the two proposed versions of the chain policy algorithm: {GPCP$_{K2}$, GPCP$_{K3}$}, with K of 2 and 3, respectively. Besides each mean value, the standard deviation is shown in the first bracket, and the results of the statistical significance test (Wilcoxon rank sum test with significance level of 0.05) are shown in the second bracket by any two of $\{+, -, =\}$ symbols. These are used to denote GPHH performance relative to each of the two chain policy methods; the first symbol refers to GPCP$_{K2}$, the second to GPCP$_{K3}$. A '$+$' indicates the corresponding GPCP algorithm was statistically significantly better than GPHH. A '$-$' indicates the opposite, and a '$=$' indicates there is no significant difference. In addition, we compare the two GPCP methods in the same manner, indicating significance between them by highlighting the superior via bolding (e.g. on $Ugdb2$, GPCP$_{K3}$ significantly outperforms GPCP$_{K2}$).

A. Observations on the Performance

From Table III, we can see that GPCP$_{K2}$ performed significantly better than GPHH on only 1 Ugdb instance (Ugdb9), while significantly worse than GPHH on 5 Ugdb instances. There was no statistically significant difference between GPCP$_{K2}$ and GPHH on the remaining 16 Ugdb instances. GPCP$_{K3}$ never outperformed GPHH on the Ugdb instances. It was significantly worse than GPHH on 5 Ugdb instances, while statistically comparable on the remaining 18 instances. Comparing GPCP$_{K2}$ and GPCP$_{K3}$, it seems like GPCP$_{K3}$ was slightly better than GPCP$_{K2}$, winning on 3 instances, while losing on 1 instance.

TABLE IV
THE TEST PERFORMANCE OF THE GPHH AND GPCP (WITH $K = 2, 3$) METHODS ON THE *val* INSTANCES.

Instance	$\|V\|$	$\|E\|$	m	GPHH	GPCP$_{K2}$	GPCP$_{K3}$
Uval1A	24	39	2	175.84(3.07)(= =)	178.14(9.97)	176.31(4.57)
Uval1B	24	39	3	184.35(1.95)(- -)	187.17(4.16)	188.63(5.28)
Uval1C	24	39	8	312.62(6.89)(= =)	314.50(9.70)	315.52(12.20)
Uval2A	24	34	2	229.49(3.26)(= -)	**230.11(3.49)**	240.82(70.49)
Uval2B	24	34	3	278.08(6.23)(= =)	275.61(5.04)	276.45(4.03)
Uval2C	24	34	8	607.35(18.60)(+ +)	595.23(17.97)	595.56(17.45)
Uval3A	24	35	2	82.33(0.94)(= -)	85.65(15.36)	83.13(1.40)
Uval3B	24	35	3	96.78(2.06)(= +)	96.49(2.46)	95.63(2.28)
Uval3C	24	35	7	174.90(4.96)(- -)	**181.03(7.79)**	184.80(7.35)
Uval4A	41	69	3	421.54(8.54)(+ =)	**419.86(14.35)**	427.54(23.59)
Uval4B	41	69	4	444.15(8.17)(= =)	478.30(205.73)	454.09(51.78)
Uval4C	41	69	5	494.78(14.76)(= =)	496.94(35.20)	493.78(11.70)
Uval4D	41	69	9	701.99(26.44)(= =)	690.33(25.25)	692.27(22.89)
Uval5A	34	65	3	443.50(4.22)(= =)	443.83(5.38)	441.21(5.09)
Uval5B	34	65	4	472.70(7.77)(= =)	516.24(238.38)	471.14(5.59)
Uval5C	34	65	5	514.58(7.62)(- -)	554.61(170.88)	**534.13(35.60)**
Uval5D	34	65	9	726.89(15.66)(= -)	**730.33(11.89)**	734.74(16.90)
Uval6A	31	50	3	230.61(9.73)(= =)	230.18(6.69)	230.37(4.11)
Uval6B	31	50	4	256.68(2.31)(= =)	264.45(43.53)	269.47(63.43)
Uval6C	31	50	10	410.48(14.23)(= =)	**406.05(9.23)**	415.43(12.91)
Uval7A	40	66	3	289.09(8.23)(= =)	287.57(6.61)	288.07(8.03)
Uval7B	40	66	4	297.40(8.65)(= =)	**299.02(19.45)**	311.65(61.45)
Uval7C	40	66	9	406.46(9.27)(= +)	407.69(10.18)	**403.28(19.03)**
Uval8A	30	63	3	398.11(3.56)(= =)	398.17(3.63)	400.38(7.10)
Uval8B	30	63	4	426.16(5.43)(= -)	**425.85(6.23)**	442.19(60.93)
Uval8C	30	63	9	669.25(18.74)(= =)	669.16(16.33)	670.26(19.84)
Uval9A	50	92	3	333.87(2.68)(= -)	347.79(78.88)	**335.52(3.73)**
Uval9B	50	92	4	348.53(3.83)(= -)	352.84(12.06)	352.53(6.30)
Uval9C	50	92	5	364.70(4.32)(+ =)	**360.96(5.46)**	365.04(4.34)
Uval9D	50	92	10	479.22(9.16)(= =)	512.88(175.73)	**477.39(7.95)**
Uval10A	50	97	3	439.40(4.25)(= -)	486.89(250.26)	**445.71(5.69)**
Uval10B	50	97	4	459.34(5.61)(+ =)	**455.69(5.58)**	463.91(21.58)
Uval10C	50	97	5	477.94(4.77)(= =)	477.20(5.08)	479.91(16.10)
Uval10D	50	97	10	621.30(5.75)(- -)	629.08(13.32)	646.02(76.55)

From Table IV, it can be seen that GPCP$_{K2}$ significantly outperformed GPHH on 4 Uval instances, while losing on 4 Uval instances. There was no statistically significant difference on the remaining 26 Uval instances. GPCP$_{K3}$ achieved significantly better, worse and comparable results with GPHH on 2, 11 and 21 Uval instances, respectively. Between GPCP$_{K2}$ and GPCP$_{K3}$, GPCP$_{K2}$ also showed better performance, significantly outperforming GPCP$_{K3}$ on 9 Uval instances, while losing on 5 instances.

Overall, we can see that:

- The GPCP algorithms did not achieve better performance than GPHH on most test instances. This suggests that it is much more challenging to use the look-ahead information for UCARP than we expected.
- The performance of the GPCP algorithms are better on the Uval instances than the Ugdb instances. This may suggest that the look-ahead information can be more useful on larger and more complex instances, which is consistent with our expectation.
- GPCP$_{K2}$ performed better than GPCP$_{K3}$. This may be because that the uncertainty increases with the increase of the chain length, making the prediction of the chain policy less accurate.

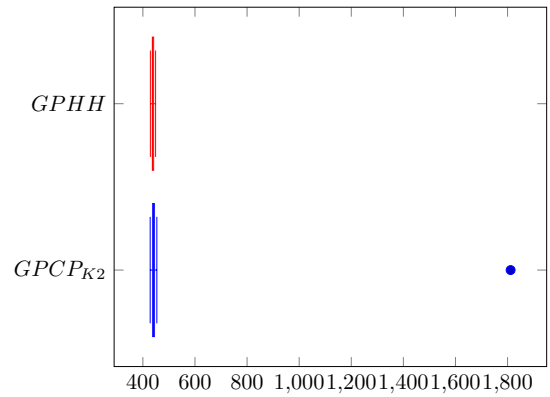

Fig. 1. The boxplot of the best rules obtained by the 30 runs of the compared algorithms on Uval10A.

In addition, we see that for some instances (e.g., Uval4B, Uval5B and Uval5C), the standard deviation of GPCP$_{K2}$ was very large. This indicates that there are some very poor outliers in the 30 runs.

B. Further Analysis

In the further analysis, we aim to a) understand why the standard deviation of the proposed GPCP methods is so high on some instances, b) investigate whether overfitting has occurred, and c) validate the effectiveness of chain selection used by GPCP.

1) High Standard Deviation: Fig. 1 shows the boxplot of the test performance obtained by the 30 runs of GPHH and GPCP$_{K2}$ on Uval10A, where GPCP$_{K2}$ had a very high standard deviation. It can be seen that GPCP$_{K2}$ had a very poor outlier with the test performance of over 1800, while that of all the other runs were less than 500. This shows that the high standard deviation on the test performance is due to the poor outlier(s). When we look into the outlier run, we found that the training performance of that run was 442.32, which was similar to other runs. This indicates that a serious overfitting may occur in this run, as the test performance was so much worse than the training performance.

2) Training vs Test Performance: Fig. 2 shows the training (x-axis) vs test performance (y-axis) obtained by the 30 runs of the compared algorithms on Uval10A. The outlier $(442.32, 1811.67)$ was omitted. From the figure, we can see that GPCP$_{K2}$ showed very similar training and test performance with GPHH. This indicates that GPCP$_{K2}$ can generalise as well as GPHH, when excluding the exceptional outlier. Other instances showed the same pattern.

3) Chain Effectiveness: If the chains help with the decision making, we expect the chain policies to a) often select the non-single-task chains (i.e. $k > 1$) and b) the first task of the selected chain is different from the most preferred single task (otherwise the decision will be the same with and without considering the chains). In addition, to keep the advantage of task chains over single tasks, the chain policy should be able to stick to the selected chains most of the time. To verify

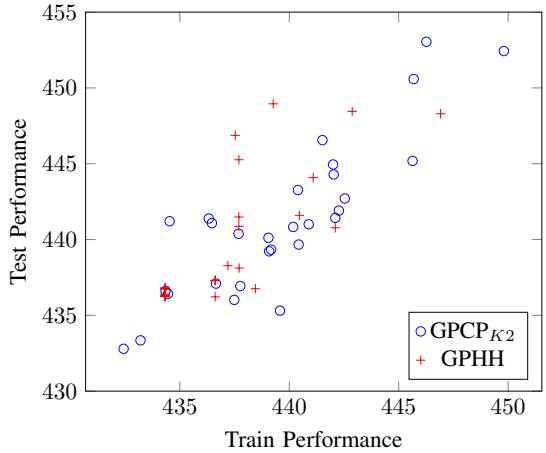

Fig. 2. The test vs training performance of the individuals for GPHH and GPCP$_{K2}$ on Uval10A. The outlier $(442.32, 1811.67)$ was excluded, in order to observe the pattern of the remaining rules.

this, we take two examples on Ugdb9 and Uval10A and 5 rules obtained by GPCP$_K$2 with different test performance, and analyse their behaviours. Table V shows the analysis results, where the column "$\ell_k = p_1$" means the percentage of decision situations where a single task is selected. "$\ell_k[0] = p_1$" ("$\ell_k[0] \neq p_1$") are the percentages of decision situations where the first task of the selected ℓ_k ($k > 1$) chain equals (differs from) the most preferred single task. "Continues to $\ell_k[1]$" means the percentage of the cases when the chain policy can and has continued its chain. In addition, the columns "#IOC" and "#ITC" are the number of times each new terminal occurred in that rule.

From the table, we have the following observations:

- All the rules in the table often selects a ℓ_2 chain (more than 42.99%).
- If a rule selects a ℓ_2 chain, the first task of the selected chain is almost always different from the best single task (no more than 3.09%).
- When comparing between the best rule and the second best rule (approx best + 10%) for the two instances, the best rules select the ℓ_2 chains less often. This suggests that it is more risky to select ℓ_2 chains, and too frequent selections of ℓ_2 chains may lead to less accurate decisions and worse test performance.
- The outlier rule for Uval10A almost never followed the selected ℓ_2 chains. This shows the importance of sticking to the selected ℓ_2 chains. If the decision process failed to stick to the selected ℓ_2 chains, then the test performance can become very poor.
- The new terminals seemed not to help the chain policies. For Ugdb9, the best rule does not use any of the new terminals. For Uval10A, the best rule used both, but for only once (out of the total 29 occurrences of the terminals in the whole tree).

4) Terminal Usage: Fig. 3 shows the frequency of using each terminal by the best rules of GPHH and GPCP$_K$2 on all

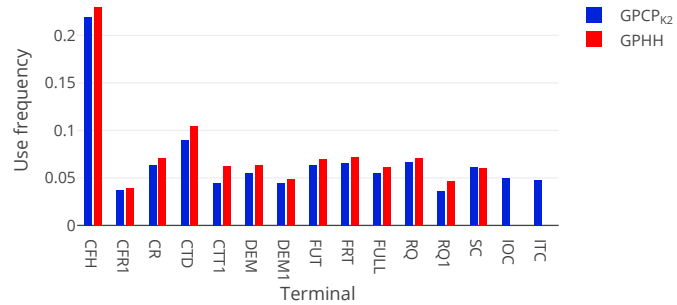

Fig. 3. The frequency of using each terminal by the best rules of GPHH and GPCP$_K$2 on all the test instances.

the test instances. The figure shows that GPCP$_K$2 considers each terminal approximately as important as GPHH (once normalised to account for the new terminals). The CFH was the most important terminal, followed by CTD. CFR1 and RQ1 were the least frequently used, as they are more complex features that are hard for GP to use.

VII. Conclusions and Future Work

The goal of this paper was to investigate the feasibility of incorporating look-ahead strategies within the reactive routing policy decision making process for UCARP. To this end, we proposed a new decision process that considers both the single tasks and task chains. This expands the candidate pool of the routing policy during the decision process, enabling it to consider the look-ahead information embedded in the task chains. We also developed two new terminals specifically for the task chains in the GPHH. The experimental studies showed some interesting results. First, the consideration of the ℓ_2 chains improved the effectiveness of the routing policy on a limited number of test instances. On most other instances, the performance was either similar or even worse. We found that in some GP runs serious overfitting can occur, leading to much worse test performance than the training performance. This may be due to the higher uncertainty in the task chains than the single tasks. Second, we observed that the test performance of the GP-evolved chain policies can depend on the balance between selecting the single tasks and the ℓ_2 chains, as well as how often it can successfully stick to the selected chains.

There are several possible future directions. First, we will develop new strategies to address the overfitting issue, and avoid the outlier runs. Second, we will improve the chain policy learning so that it can select better and more robust chains, and can stick to the selected chains more often. We will also develop new strategies that can use the chain-related new terminals more effectively.

References

[1] B. Golden and R. Wong, "Capacitated arc routing problems," *Networks*, vol. 11, no. 3, pp. 305–315, 1981.

[2] B. Golden, J. Dearmon, and E. Baker, "Computational experiments with algorithms for a class of routing problems," *Computers & Operations Research*, vol. 10, pp. 47 – 59, 1983.

TABLE V
THE STATISTICAL BEHAVIOUR OF THE SELECTED RULES FOR UGDB9 AND UVAL10A.

Identifier	**Test Performance**	**#IOC**	**#ITC**	$\ell_k = p_1$	$\ell_k[0] = p_1$	$\ell_k[0] \neq p_1$	**Continues to** $\ell_k[1]$
$gdb9$: best test rule	369.95	0	0	56.86%	0.15%	42.99%	39.28%
$gdb9$: approx best +10%	406.01	1	0	26.75%	1.26%	71.99%	45.64%
$val10A$: best test rule	432.79	1	1	30.79%	3.09%	66.11%	72.70%
$val10A$: approx best +10%	476.30	0	0	14.89%	2.80%	82.31%	61.73%
$val10A$: worst test rule	1811.67	1	0	36.34%	0.00%	63.66%	1.22%

[3] M. C. Mourão and M. T. Almeida, "Lower-bounding and heuristic methods for a refuse collection vehicle routing problem," *European Journal of operational research*, vol. 121, no. 2, pp. 420–434, 2000.

[4] S. K. Amponsah and S. Salhi, "The investigation of a class of capacitated arc routing problems: the collection of garbage in developing countries," *Waste management*, vol. 24, no. 7, pp. 711–721, 2004.

[5] S. Wøhlk and G. Laporte, "A districting-based heuristic for the coordinated capacitated arc routing problem," *Computers & Operations Research*, vol. 111, pp. 271–284, 2019.

[6] X. Jin, H. Qin, Z. Zhang, M. Zhou, and J. Wang, "Planning of garbage collection service: An arc-routing problem with time-dependent penalty cost," *IEEE Transactions on Intelligent Transportation Systems*, 2020.

[7] M. Çelik, Ö. Ergun, and P. Keskinocak, "The post-disaster debris clearance problem under incomplete information," *Operations Research*, vol. 63, no. 1, pp. 65–85, 2015.

[8] V. Akbari and F. S. Salman, "Multi-vehicle synchronized arc routing problem to restore post-disaster network connectivity," *European Journal of Operational Research*, vol. 257, no. 2, pp. 625–640, 2017.

[9] ——, "Multi-vehicle prize collecting arc routing for connectivity problem," *Computers & Operations Research*, vol. 82, pp. 52–68, 2017.

[10] H. Handa, L. Chapman, and X. Yao, "Robust route optimization for gritting/salting trucks: A cercia experience," *IEEE Computational Intelligence Magazine*, vol. 1, no. 1, pp. 6–9, 2006.

[11] M. Tagmouti, M. Gendreau, and J.-Y. Potvin, "A dynamic capacitated arc routing problem with time-dependent service costs," *Transportation Research Part C: Emerging Technologies*, vol. 19, no. 1, pp. 20–28, 2011.

[12] A. H. Gundersen, M. Johansen, B. S. Kjær, H. Andersson, and M. Stålhane, "Arc routing with precedence constraints: An application to snow plowing operations," *Computational Logistics*, p. 174, 2017.

[13] Y. Mei, K. Tang, and X. Yao, "Capacitated arc routing problem in uncertain environments," in *IEEE Congress on Evolutionary Computation*. IEEE, 2010, pp. 1–8.

[14] Y. Mei and M. Zhang, "Genetic programming hyper-heuristic for multi-vehicle uncertain capacitated arc routing problem," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. ACM, 2018, pp. 141–142.

[15] J. Wang, K. Tang, J. A. Lozano, and X. Yao, "Estimation of the distribution algorithm with a stochastic local search for uncertain capacitated arc routing problems," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 1, pp. 96–109, 2016.

[16] T. Weise, A. Devert, and K. Tang, "A developmental solution to (dynamic) capacitated arc routing problems using genetic programming," in *Proceedings of the 14th annual conference on Genetic and evolutionary computation*. ACM, 2012, pp. 831–838.

[17] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "Automated heuristic design using genetic programming hyper-heuristic for uncertain capacitated arc routing problem," in *Proceedings of the Genetic and Evolutionary Computation Conference*. ACM, 2017, pp. 290–297.

[18] J. MacLachlan, Y. Mei, J. Branke, and M. Zhang, "Genetic programming hyper-heuristics with vehicle collaboration for uncertain capacitated arc routing problems," *Evolutionary computation*, pp. 1–29, 2019.

[19] P. Lacomme, C. Prins, and W. Ramdane-Cherif, "Competitive memetic algorithms for arc routing problems," *Annals of Operations Research*, vol. 131, no. 1-4, pp. 159–185, 2004.

[20] H. Al-Sahaf, Y. Bi, Q. Chen, A. Lensen, Y. Mei, Y. Sun, B. Tran, B. Xue, and M. Zhang, "A survey on evolutionary machine learning," *Journal of the Royal Society of New Zealand*, vol. 49, no. 2, pp. 205–228, 2019.

[21] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "A predictive-reactive approach with genetic programming and cooperative coevolution for the uncertain capacitated arc routing problem," *Evolutionary computation*, pp. 1–28, 2019.

[22] C. C. Perez, *Invisible women: Exposing data bias in a world designed for men*. Random House, 2019.

[23] C. D. J. Waters, "Vehicle-scheduling problems with uncertainty and omitted customers," *Journal of the Operational Research Society*, vol. 40, no. 12, pp. 1099–1108, 1989.

[24] B. F. Moghaddam, R. Ruiz, and S. J. Sadjadi, "Vehicle routing problem with uncertain demands: An advanced particle swarm algorithm," *Computers & Industrial Engineering*, vol. 62, no. 1, pp. 306–317, 2012.

[25] L. Wu, M. Hifi, and H. Bederina, "A new robust criterion for the vehicle routing problem with uncertain travel time," *Computers & Industrial Engineering*, vol. 112, pp. 607–615, 2017.

[26] D. Ouelhadj and S. Petrovic, "A survey of dynamic scheduling in manufacturing systems," *Journal of scheduling*, vol. 12, no. 4, p. 417, 2009.

[27] S. Nguyen, Y. Mei, H. Ma, A. Chen, and M. Zhang, "Evolutionary scheduling and combinatorial optimisation: Applications, challenges, and future directions," in *2016 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2016, pp. 3053–3060.

[28] S. Gonzalez-Martin, A. A. Juan, D. Riera, M. G. Elizondo, and J. J. Ramos, "A simheuristic algorithm for solving the arc routing problem with stochastic demands," *Journal of Simulation*, vol. 12, no. 1, pp. 53–66, 2018.

[29] S. González, D. Riera, A. A. Juan, M. G. Elizondo, and P. Fonseca, "Sim-randsharp: A hybrid algorithm for solving the arc routing problem with stochastic demands," in *Proceedings of the 2012 Winter Simulation Conference*, 2012, pp. 1–11.

[30] N. Toklu, R. Montemanni, and L. M. Gambardella, "An ant colony system for the capacitated vehicle routing problem with uncertain travel costs," in *2013 IEEE Symposium on Swarm Intelligence (SIS)*. IEEE, 2013, pp. 32–39.

[31] M. A. Ardeh, Y. Mei, and M. Zhang, "A novel genetic programming algorithm with knowledge transfer for uncertain capacitated arc routing problem," in *Pacific Rim International Conference on Artificial Intelligence*. Springer, 2019, pp. 196–200.

[32] ——, "Genetic programming hyper-heuristic with knowledge transfer for uncertain capacitated arc routing problem," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2019, pp. 334–335.

[33] S. Wang, Y. Mei, and M. Zhang, "Novel ensemble genetic programming hyper-heuristics for uncertain capacitated arc routing problem," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2019, pp. 1093–1101.

[34] S. Wang, Y. Mei, J. Park, and M. Zhang, "Evolving ensembles of routing policies using genetic programming for uncertain capacitated arc routing problem," *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2019.

[35] L. Bertazzi and N. Secomandi, "Faster rollout search for the vehicle routing problem with stochastic demands and restocking," *European Journal of Operational Research*, vol. 270, no. 2, pp. 487–497, 2018.

[36] M. W. Ulmer, J. C. Goodson, D. C. Mattfeld, and M. Hennig, "Offline–online approximate dynamic programming for dynamic vehicle routing with stochastic requests," *Transportation Science*, 2018.

[37] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "A predictive-reactive approach with genetic programming and cooperative coevolution for the uncertain capacitated arc routing problem," *Evolutionary computation*, vol. 28, no. 2, pp. 289–316, 2020.

[38] N. Pillay and R. Qu, *Hyper-heuristics: theory and applications*. Springer, 2018.